

Namespaces Without a Name Server

Contraction, copying, and a fully abstract encoding of the π -calculus in the rho calculus

L. G. Meredith*

August 2026

Abstract

The rho calculus was designed so that its theory of names would be *internal*: names are quoted processes, there is no name-restriction operator, and no ambient supply of atoms. The encoding of the π -calculus given with the original presentation was meant to demonstrate that ν can be discharged under that constraint — that one need not posit a runtime source of fresh names. Lybech showed that the encoding is wrong, by two counterexamples, and gave a corrected encoding that is provably valid against Gorla-style criteria. His correction obtains freshness from a single central *name server* contacted by every ν in the term. That is a correct encoding of a different thing: it discharges ν by supposing precisely the resource whose elimination was the point.

We repair the encoding without a name server. The repair is not an invention. The original construction was modelled on Abramsky’s computational interpretation of classical linear logic, and used only half of it: the multiplicative *splitting* of a namespace, and not the *rejoining* performed by the Contraction term former $t @ u$ together with the Copy reaction rule. Abramsky’s Copy carries an invariant — names are pairs $\langle x_0, s \rangle$ with s a binary address, and two names sharing a root have *incompatible* addresses — which is exactly the invariant the original encoding needed and never stated. We show why the missing half is not optional in the rho calculus: Copy renames the contents of a box at the metalevel, whereas rho’s duplicator copies quoted code verbatim, and rho substitution does not descend under a quote. Address arithmetic must therefore move out of the syntax and into processes. We give the resulting encoding, in which each replication carries its own local address server derived from its own address, verify it against both of Lybech’s counterexamples, and prove it fully abstract with respect to a bisimilarity whose observers are restricted to the *encodings of source contexts*. We show that this restriction is forced rather than convenient: an unrestricted rho context separates the encodings of P and $P \mid \mathbf{0}$.

Throughout we use the current concrete syntax for the rho calculus — $\text{for}(y \leftarrow x)P$, $x!(P)$, $@P$, $*x$ — and explain why it, rather than the corner-quote notation of the original papers, is the notation in which the central lemma of this note can be stated at all.

1 Introduction

Process calculi customarily begin by positing a countably infinite set of atomic names, and a scoping operator $(\nu x)P$ which conjures a name guaranteed fresh. Both are idealisations, and both are discharged somewhere below the model: an implementation must say where names come from and how freshness is achieved, and in a distributed setting the answer is not obvious.

*F1R3FLY.io. lgreg.meredith@f1r3fly.io

The rho calculus [11] was an attempt to move that question inside the theory. Names are quoted processes; the only binder is input; there is no ν . Freshness, when it is available, is a consequence of structural distinctness of the processes quoted, not of an oracle. Whether this is a viable position is exactly the question of whether the π -calculus can be encoded in the rho calculus *without reintroducing what was removed*, and the original paper offered an encoding to that end. The price paid was that the translation is not a homomorphism on a signature: it compiles a whole term at a time, because the namespace discipline is a property of the term’s structure.

Lybech [10] showed that the encoding is incorrect.¹ He exhibits two counterexamples, diagnoses their common cause precisely, proposes a new encoding, and proves it valid against a list of criteria close to Gorla’s [6], augmented by a criterion of *parameter independence* which is the right addition for parametrised translations. All of this is correct, and this note does not dispute any of it.

What this note disputes is the repair. Lybech’s encoding obtains fresh names from a process

$$!N(x, z, v, s)$$

— a name server — reached over a fixed channel v which every occurrence of ν in the entire term must contact. He is candid about the cost, and closes his paper by observing that a more elaborate encoding, in which each replication instantiates its own source of names, “would be closer to the intention in the encoding by Meredith and Radestock”, but that it appears to face a *scaffolding* problem: to build a source of fresh names one must already have a source of fresh names. He leaves it open.

The scaffolding problem has a solution, and it was in the construction the original encoding was modelled on. Abramsky’s proof expressions for classical linear logic [1] handle exactly this: unboundedly many copies of a proof, each needing names distinct from every other copy’s, with no fresh names manufactured at runtime at all. The mechanism is that a name is a pair $\langle x_0, s \rangle$ of a root and a binary *address*; the Copy rule extends addresses rather than allocating roots; and Copy’s own first clause *re-contracts* the copied box’s side ports, so that the parent’s namespace is never disturbed. Freshness is required only where roots are introduced — statically, finitely often, at translation time.

The original encoding used the first half of this. Its splitting of the parameter pair across parallel composition, $\llbracket P_1 \mid P_2 \rrbracket_{n,p} = \llbracket P_1 \rrbracket_{n+,p} \mid \llbracket P_2 \rrbracket_{n+,p+}$, is precisely Abramsky’s variant operation $t \mapsto t^l, t^r$, and it is correct for precisely his reason. It did not use the second half, and what the second half supplies is not decoration: it is the reason the invariant survives copying.

Contributions.

- (i) We identify the invariant the 2005 encoding required and never stated (§6), and show that the invariant Lybech reverse-engineers from it — that the parameters are the most recently replicated names — is a different and weaker property that no splitting clause can preserve, whereas Abramsky’s prefix-incompatibility invariant is preserved by splitting by construction. On this reading the parallel composition clause is not the defect; it is the half that was right.

¹The author discussed this with Lybech in early 2023 and mentioned that Abramsky’s construction [1] was the source of the encoding’s design and, in the author’s view, the source of its repair; the thread was not taken up. The present note makes the argument in full.

- (ii) We prove a no-go result (Proposition 6.5) explaining why having the right invariant is still not enough: a translation function is a finite object and can emit only finitely many quoted addresses, while replication yields unboundedly many occurrences; and rho's duplicator cannot make up the difference, because it copies quoted code verbatim and rho substitution does not descend under a quote. Address arithmetic must therefore be performed by communication. This is stated as a syntactic side condition on translation clauses (Definition 6.6), which both of Lybech's counterexamples violate by inspection.
- (iii) We give an encoding satisfying that condition (§7), in which each replication node carries a local address server derived from its own address, the scaffolding regress terminates at a static seed consumed once, and no process is contacted by more than one node. Whole-term compilation survives: the address structure is still written statically and still mirrors the parse tree; only the addresses themselves become runtime values.
- (iv) We verify the encoding against both of Lybech's counterexamples (§8), adapted to the input-guarded fragment.
- (v) We prove full abstraction (§9) for a context-labelled bisimilarity whose observers are the encodings of source contexts, and we show (§9.5) that the restriction is forced: an unrestricted rho context separates $\llbracket P \rrbracket$ from $\llbracket P \mid \mathbf{0} \rrbracket$ although $P \equiv P \mid \mathbf{0}$. Lybech's own separation theorem is untouched and explains why no encoding whatever can do better.

Scope. The source language is the asynchronous, choice-free π -calculus with *input-guarded* replication. This is not a concession extracted by the difficulty: unguarded replication is nonsense from an implementation standpoint — it runs away — and the 2005 encoding of it diverges unconditionally, as Lybech notes. He restricts to the same fragment for the same reason, and observes (his footnote 5) that the restriction would not by itself have prevented either of his counterexamples. Cost accounting is out of scope. This note is a bug fix.

2 The rho calculus

We recall the calculus [11], in current notation.

Definition 2.1 (Syntax). Processes P and names x are given by mutual recursion:

$$P ::= \mathbf{0} \mid P_1 \mid P_2 \mid x!(P) \mid \text{for}(y \leftarrow x)P \mid *x \qquad x ::= @P$$

$x!(P)$ is the *lift*: it quotes P , thereby creating the name $@P$, and outputs it on x . $\text{for}(y \leftarrow x)P$ is the input, binding y in P ; it is the only binder. $*x$ is the *drop*: it strips the quotes from x and runs the process inside.

Definition 2.2 (Name equivalence and structural congruence). Name equivalence $\equiv_{\mathcal{N}}$ is the least equivalence on names closed under

$$\frac{P_1 \equiv P_2}{@P_1 \equiv_{\mathcal{N}} @P_2} \qquad \frac{x_1 \equiv_{\mathcal{N}} x_2}{@(*x_1) \equiv_{\mathcal{N}} x_2}$$

and structural congruence $\equiv$ is the least congruence containing α -equivalence and the abelian monoid laws for $\mid$ with unit $\mathbf{0}$, with names compared up to $\equiv_{\mathcal{N}}$ in the congruence rules.

Definition 2.3 (Reduction).

$$\frac{x_1 \equiv_{\mathcal{N}} x_2}{\text{for}(y \leftarrow x_1)P_1 \mid x_2!(P_2) \rightarrow P_1\{\text{@}P_2/y\}}$$

closed under $\mid$ and $\equiv$ in the usual way.

Substitution is capture-avoiding and replaces all names $\equiv_{\mathcal{N}}$ -equal to the substituted variable. Two clauses matter for everything below:

$$(\text{@}P)\{x/y\} = \begin{cases} x & \text{if } \text{@}P \equiv_{\mathcal{N}} y \\ \text{@}P & \text{otherwise} \end{cases} \quad (*x)\{\text{@}P/y\} = \begin{cases} P & \text{if } x \equiv_{\mathcal{N}} y \\ *x & \text{otherwise} \end{cases}$$

The first is the crux of this paper: *substitution does not recur into a process under a quote*. A quoted process is an atom to substitution; it is replaced wholesale or not at all.

Notation 2.4. We write $x!(*z)$ for the transmission of the *name* z along x , since $\text{@}(*z) \equiv_{\mathcal{N}} z$; and we write $D(x)$ for the duplicator $\text{for}(y \leftarrow x)(*y \mid x!(*y))$, which satisfies $x!(Q) \mid D(x) \rightarrow Q \mid x!(Q)$.

2.1 Why this notation

The 2005 papers wrote input as $x(y).P$ and quotation with corner quotes. We use $\text{for}(y \leftarrow x)P$, $\text{@}P$, $*x$ throughout, for four reasons, of which the last is the one that earns its place in a paper about a bug.

- (1) It is ASCII. Every term displayed in this note is a term of rho-lang; the appendix runs.
- (2) It extends to joins without renegotiation. Simultaneous consumption from several channels is written $\text{for}(y_1 \leftarrow x_1 \ \& \ y_2 \leftarrow x_2)P$, with $\&$ joining within a group and $;$ separating alternative groups. The corner-quote prefix notation $x(y).P$ has no comparable extension: the prefix is a unary constructor by construction, and polyadic or joined receipt has to be simulated. The encoding of §7 does not require joins, but the generalisation of §12 to polyadic π does, and it is better to have paid for the notation in advance.
- (3) It is the for-comprehension of Scala, Haskell’s **do**, and C#’s LINQ. A reader who knows none of the literature on process calculi nonetheless reads $\text{for}(y \leftarrow x)P$ correctly on the first attempt: draw a y from x , then do P . The monadic reading of input is not an analogy here; RSpace, the store underlying the rho calculus implementations, is a comprehension over a tuplespace.
- (4) Most importantly, $\text{@}\cdot$ and $*\cdot$ are visibly *operators applied to terms*, occupying positions in a term. The central technical observation of this note (Lemma 6.3) is a statement about which *positions* substitution can reach: it reaches the object of a lift, which is a process position, and it does not reach beneath a quote. Corner quotes present quotation as a change of typographic register rather than as an operator, and in that notation the distinction between $cl(n!(\mathbf{0}))$ and $\text{@}n!(\mathbf{0})$ — which is the entire difference between the broken encoding and the repaired one — is typographically almost invisible. It was, in fact, invisible for seventeen years.

Remark 2.5. The calculus is the **r**eflective **h**igher **o**rders calculus; “rho” is an acronym before it is a Greek letter, and the pun — that rho follows π — is the point of the name. We therefore never write it with the letter ρ .

3 The 2005 encoding and what it was for

Fix an injection φ from π -calculus names to rho names; we suppress it. The *static quoting* techniques generate, from a name x , the names

$$\mathbf{L}(x) \triangleq @ (x! (\mathbf{0})) \quad \mathbf{R}(x) \triangleq @ (\text{for}(y \leftarrow x) \mathbf{0})$$

(the left and right increments, written $+x$ and $x+$ in the original), and their composition. Each generates a countably infinite sequence of names from any starting point.

The translation $\llbracket - \rrbracket_{n,p}$ takes two name parameters, and is defined by

$$\begin{aligned} \llbracket \mathbf{0} \rrbracket_{n,p} &= \mathbf{0} \\ \llbracket x(y).P \rrbracket_{n,p} &= \text{for}(y \leftarrow x) \llbracket P \rrbracket_{n,p} \\ \llbracket \bar{x}(z) \rrbracket_{n,p} &= x!(*z) \\ \llbracket P_1 \mid P_2 \rrbracket_{n,p} &= \llbracket P_1 \rrbracket_{\mathbf{L}(n), \mathbf{L}(p)} \mid \llbracket P_2 \rrbracket_{\mathbf{R}(n), \mathbf{R}(p)} \\ \llbracket (\nu z)P \rrbracket_{n,p} &= \text{for}(z \leftarrow p) \llbracket P \rrbracket_{\mathbf{L}(n), \mathbf{L}(p)} \mid p!(*n) \\ \llbracket !P \rrbracket_{n,p} &= a!((\text{for}(n \leftarrow b) \text{for}(p \leftarrow c) (\llbracket P \rrbracket_{n,p} \mid \mathbf{D}(a) \mid \dots))) \mid \mathbf{D}(a) \mid \dots \end{aligned}$$

with $a = n \cdot p$, $b = \mathbf{R}(n)$, $c = \mathbf{R}(p)$, and top-level parameters $n_0 = @ \prod_{x \in \text{fn}(P)} x! (\mathbf{0})$, $p_0 = @ \prod_{x \in \text{fn}(P)} \text{for}(y \leftarrow x) \mathbf{0}$, chosen so that no name of P is derivable from them.

Two features of this design should be recorded before it is criticised, because they are what the design is *for*.

- (a) **There is no runtime source of names.** n_0 and p_0 are computed by the translation from $\text{fn}(P)$ and are never requested from anything. Every other name in the image is obtained from them by operations that are, in intent, local to the node that performs them.
- (b) **The translation compiles a whole term at a time.** It is parametrised by a position in the term, not merely by the term's head constructor. This is the price of (a), and it is the reason the encoding is not a signature homomorphism.

The defect, stated in the way we will need it, is this: the encoding carries an invariant on (n, p) that is never written down. Lybech recovers it as *the parameters are the names most recently replicated*, and this is a fair reconstruction of what the replication clause is trying to maintain. We shall argue in §6 that it is the wrong invariant, and that the encoding's own ancestry supplies the right one.

4 Lybech's counterexamples and his repair

4.1 The counterexamples

Example 4.1 (Splitting destroys the parameter invariant). Let Q be arbitrary and

$$P'_1 \triangleq !((\nu z) \bar{u}(z) \mid Q) \quad P'_2 \triangleq (\nu z) !(\bar{u}(z) \mid Q).$$

P'_1 emits a different fresh name on u at each unfolding; P'_2 emits the same one forever, and the context $[\cdot] \mid u(n_1).u(n_2).(\bar{n}_1 \mid n_2.\bar{x})$ distinguishes them. Their images do not differ: the replication context rebinds n and p at each unfolding, but the parallel composition clause has already frozen $\mathbf{L}(n)$ and $\mathbf{L}(p)$ *inside quotes*, where the runtime substitution cannot reach, so the inner ν emits the same name every time.

Example 4.2 (Nesting ν destroys it too).

$$P_1 \triangleq !(\nu z)\bar{u}\langle z \rangle \quad P_2 \triangleq !(\nu q)(\nu z)\bar{u}\langle z \rangle$$

are structurally congruent, since q is unused. Their images are not equivalent: the clause for (νq) statically increments the parameters before the clause for (νz) sees them, so in $\llbracket P_2 \rrbracket$ the emitted names are all equal while in $\llbracket P_1 \rrbracket$ they are all distinct.

Lybech’s diagnosis is exact, and we adopt it verbatim: the property of being “the most recently replicated names” is not preserved by the translation of parallel composition, because the increments the translation writes are *static*, i.e. they place the parameter under a quote, and substitution does not recur under quotes.

Remark 4.3. Both examples use unguarded replication, which our fragment excludes. The errors survive the restriction — Lybech records as much explicitly — and we restate the examples in the input-guarded fragment in §8, where we check the repaired encoding against them.

4.2 The name server

Lybech’s encoding is $\llbracket P \rrbracket \triangleq \llbracket P \rrbracket_{n,v} \mid !N(x, z, v, s)$ where the server

$$!N(x, z, v, s) \triangleq D(x) \mid x!((\text{for}(a \leftarrow z)\text{for}(r \leftarrow v)(D(x) \mid r!(*a) \mid z!(a!(\mathbf{0})))) \mid z!(*s))$$

holds a seed on z , and answers requests arriving on the fixed channel v with successive left increments of s . The parameter n is threaded forward through the translation and is never rebound, which is what removes the second error; the first is removed because access to the name source is by a channel v that never changes and so can never be lost. The encoding is proved valid against compositionality, substitution invariance, operational correspondence, observational correspondence, divergence reflection, and parameter independence. We accept all of this.

We also note the mechanism the server uses to build the successor name:

$$z!(a!(\mathbf{0}))$$

where a is a name that was *received*. This computes $L(a)$ at runtime, and it works because the object of a lift is a process position. It is the very manoeuvre the broken clauses needed and did not use. Lybech deploys it once, inside a server, without remarking that it is a general discipline; §6 makes it one.

4.3 Why this is a correct encoding of a different thing

- (1) **It reintroduces the resource whose elimination was the point.** A single process, reached over a single fixed channel, is the source of every fresh name in the term. That is the π -calculus assumption of an ambient name supply, implemented rather than discharged. It is not a criticism of the theorem; it is an observation about what the theorem is about.
- (2) **The topology is wrong.** Lybech makes this criticism himself, and better than we would: for a distributed system with a topology other than a star, the translation does not yield an adequate representation of the source, because it routes every name request through one node. The encoding preserves the semantics of a program and not the intuitions underlying its structure.

- (3) **The criteria are not the right yardstick here.** A Gorla-style list with compositionality as a primitive measures how close a translation is to a signature homomorphism. But behavioural faithfulness and syntactic homomorphism are independent: Milner’s encoding of the λ -calculus into π is behaviourally faithful and is not a signature homomorphism. Since compiling a whole term at a time is precisely what the rho encoding trades for locality, adopting compositionality as a primitive criterion rules the interesting design out by fiat. In §9 we take the correctness criterion to be a single behavioural condition and derive Lybech’s criteria from it.

Finally, his own conclusion states the alternative and the obstacle: give each replication its own copy of the name server, “closer to the intention” of the original, and confront the scaffolding problem, since to instantiate a source of fresh names one must first have a source of fresh names. This note answers that. The answer was available in 1993.

5 The missing half: Abramsky’s construction

Abramsky’s computational interpretation of classical linear logic [1] assigns *proof expressions* $\Theta; \bar{t}$ to sequent proofs and gives them an operational semantics in the style of the chemical abstract machine. We need four features of it.

Names carry addresses. In the notational interlude preceding the reaction rules, Abramsky fixes a bijection

$$\mathcal{N} \cong \mathbb{N}_0 \times \{l, r\}^*$$

so that a name is a pair $\langle x_0, s \rangle$ of a *root* and a binary *address*. For a term t , the variants t^l and t^r replace every $\langle x_0, s \rangle$ occurring in t by $\langle x_0, sl \rangle$ and $\langle x_0, sr \rangle$ respectively. The invariant established by the proof-expression assignment and maintained by every transition is:

for all distinct names $\langle x_0, s \rangle, \langle y_0, t \rangle$ occurring in P , $x_0 = y_0$ implies that s and t are *incompatible* — they have no upper bound in the prefix ordering.

Contraction is a term former. The Contraction rule of PE_2 is

$$\frac{\vdash \Theta; \Gamma, t : ?A, u : ?A}{\vdash \Theta; \Gamma, t @ u : ?A}$$

The joiner $t @ u$ is a term, not a rewrite: it is a first-class object that can be embedded in larger terms and cut against other terms.²

Copy is the rejoin. The reaction rule for the exponential is

$$\bar{x}(P) \perp u @ v \longrightarrow \bar{x} \perp (\bar{x}^l @ \bar{x}^r), \quad \bar{x}(P)^l \perp u, \quad \bar{x}(P)^r \perp v.$$

Three things are worth separating here.

²The rule is printed in [1] with $u : ?B$; comparison with the intuitionistic Contraction rule and with case (12) of the realizability proof shows this to be a typo for $?A$.

- (i) Copying is *demand-driven*: the box duplicates only when a contraction node asks for two. Abramsky says as much — the term of type $?A^\perp$ specifies how many copies of the $!A$ term are required — and records that Copy is the only rule that increases the size of a proof expression.
- (ii) Distinctness of the two copies is by *address extension*, not by allocation. No fresh root is created. Freshness is required only where roots are introduced, and roots are introduced only by the name convention at the Axiom, With and Of Course rules — statically, finitely often.
- (iii) The first clause, $\bar{x} \perp (\bar{x}^l @ \bar{x}^r)$, is the rejoin. The box’s side ports $\bar{x}$ are cut against a *newly built contraction* of the two copies’ corresponding ports. The environment continues to see one port; the two copies’ demands on it are routed through a joiner. *The parent’s namespace is never re-rooted.*

The correspondence.

PE ₂	2005 encoding	status
$\mathcal{N} \cong \mathbb{N}_0 \times \{l, r\}^*$	L, R namespace templates	used
variants t^l, t^r	$\llbracket P_1 \mid P_2 \rrbracket_{n,p} = \llbracket P_1 \rrbracket_{L(n), L(p)} \mid \llbracket P_2 \rrbracket_{R(n), R(p)}$	used
prefix-incompatibility invariant	—	not stated
Contraction $t @ u$	—	unused
Copy, clause $\bar{x} \perp (\bar{x}^l @ \bar{x}^r)$	—	unused
demand-driven copying	eager replication (diverges)	unused

6 Diagnosis

Proposition 6.1 (The reconstructed invariant is the wrong one). *Let $I(n, p)$ be the property that (n, p) are the names most recently produced by the enclosing replication context. No splitting clause preserves I : if $\llbracket P_1 \mid P_2 \rrbracket_{n,p}$ delegates to sub-translations at parameters derived from (n, p) , then the derived parameters do not satisfy I in the sub-terms, since they are not the most recent output of the replication context but a function of it.*

Proof. Immediate from the shape of I : it is a statement about a moving frontier, and it identifies one distinguished pair globally, whereas splitting produces two pairs neither of which is that one. It follows that any repair which preserves I must avoid deriving parameters at $\mid$ — which is what Lybech’s threading of a single n through the translation does, at the cost of needing an external source for the values n takes. $\square$

Proposition 6.2 (Prefix-incompatibility is preserved by splitting). *Let $\text{Inc}(S)$ be the property that the addresses occurring in a set S of names over a common root are pairwise incompatible in the prefix ordering. If $\text{Inc}(S)$ and $\sigma \in S$, then $\text{Inc}((S \setminus \{\sigma\}) \cup \{\sigma l, \sigma r\})$.*

Proof. σl and σr are incompatible with each other. For $\tau \in S \setminus \{\sigma\}$: τ is incompatible with σ , hence neither is a prefix of the other, hence τ is incompatible with every extension of σ . $\square$

Propositions 6.1 and 6.2 together say that the parallel composition clause of the 2005 encoding is not the defect. Against the invariant its ancestry supplies, splitting is exactly the operation that maintains it. The defect is elsewhere.

Lemma 6.3 (Substitution transparency). *The object of a lift is a substitution-transparent position and a quote is not: for all P, x, y ,*

$$(c!(P))\{x/y\} = c\{x/y\}!((P\{x/y\})) \quad \text{whereas} \quad (@P)\{x/y\} \in \{x, @P\}.$$

Consequently a name may be derived from a runtime-bound parameter n by the process $c!(n!(\mathbf{0}))$, which transmits $L(n)$ on c ; and it may not be derived by writing $@(n!(\mathbf{0}))$ in the translation.

Proof. By the definition of substitution: the lift’s object is a process, and substitution is defined by structural recursion through process constructors; quotation is not a process constructor but the name constructor, and its clause is the two-case clause above. $\square$

Corollary 6.4 (One quote level per rendezvous). *A name at quote depth k above a runtime-received name requires at least k communications to construct.*

Proposition 6.5 (No static address assignment). *There is no translation function $\llbracket - \rrbracket$ of the 2005 shape — finitely many clauses, each emitting finitely many quoted names — for which the address carried by a name in the k -th unfolding of a replication is written by the translation, for all k .*

Proof. The translation is a finite object, so the set of quoted names appearing syntactically in the image of a fixed term is finite. A replication yields unboundedly many occurrences of its body, each of which must, if the body contains ν , contribute a name distinct from all the others. The pigeonhole principle applies. The only remaining hope is that substitution, applied at each unfolding, differentiates copies of a fixed piece of syntax — but by Lemma 6.3 substitution cannot reach beneath a quote, and by construction rho’s duplicator reproduces the quoted body verbatim. This is not an accident of the duplicator: transmission of code without modification is what reflection *is*, and it is the same fact on which Lybech’s separation theorem turns. $\square$

Proposition 6.5 is the precise sense in which Abramsky’s Copy rule does not transfer. Copy renames the contents of a box at the metalevel; rho has no metalevel operation on quoted code. Address arithmetic must therefore leave the syntax and become communication, and the structure that routes it is Copy’s other clause: the shared port, re-contracted per copy.

Definition 6.6 (The discipline). A translation clause is *admissible* if every parameter that will be bound at runtime occurs in it only

- (a) in subject position of an input or a lift, or
- (b) within the object of a lift,

and never within a quote written by the translation.

Proposition 6.7. *The clauses for $|$ and for ν in the 2005 encoding are inadmissible; those for $\mathbf{0}$, input and output are admissible.*

Proof. By inspection: $L(n) = @(n!(\mathbf{0}))$ places the parameter under a quote written by the translation, and both clauses use it. The remaining clauses do not mention the parameters except by passing them along. $\square$

This is the whole of the repair, and it is checkable in a minute. Both of Lybech’s counterexamples exploit exactly the two inadmissible clauses.

7 The encoding

7.1 Addresses

Definition 7.1 (Address templates). For a name σ put $L(\sigma) = @(\sigma!(\mathbf{0}))$ and $R(\sigma) = @(\text{for}(y \leftarrow \sigma)\mathbf{0})$. An *address* is a name of the form $\theta(n_0)$ for $\theta \in \{L, R\}^*$ and n_0 the static seed.

We never write L or R in the image of the translation; they are metalanguage, used only to *name* the addresses that the encoding produces by communication.

Lemma 7.2 (Injectivity and disjointness). *L and R are injective on names up to $\equiv_{\mathcal{N}}$, their images are disjoint, and $\text{qd}(L(\sigma)) = \text{qd}(R(\sigma)) = 1 + \text{qd}(\sigma)$, where qd is Lybech's quote depth. Consequently $\theta \mapsto \theta(n_0)$ is injective on $\{L, R\}^*$.*

Proof. Injectivity: $@(\sigma!(\mathbf{0})) \equiv_{\mathcal{N}} @(\tau!(\mathbf{0}))$ requires $\sigma!(\mathbf{0}) \equiv \tau!(\mathbf{0})$, hence $\sigma \equiv_{\mathcal{N}} \tau$; similarly for R . Disjointness: a lift and an input are distinct constructors and structural congruence does not relate them. Neither template's output is of the form $@(*x)$, so [N-DROP] does not apply and quote depth strictly increases. Injectivity of $\theta \mapsto \theta(n_0)$ follows by induction on $|\theta|$, using disjointness at each step and stratification to prevent an address from coinciding with a proper prefix of itself. $\square$

Lemma 7.2 is Abramsky's bijection $\mathcal{N} \cong \mathbb{N}_0 \times \{l, r\}^*$, realised inside the rho calculus with n_0 as the unique root. Prefix-incompatibility becomes, here, the conjunction of two things already in the literature: stratification by quote depth separates addresses of different length, and template disjointness separates siblings.

7.2 The one primitive

Everything is built from a single macro, which obtains two fresh addresses from a given one by communication.

Construction 7.3 (Fresh_2).

$$\text{Fresh}_2(\sigma; u, v).P \triangleq \text{for}(u \leftarrow \sigma)\text{for}(v \leftarrow \sigma)P \mid \sigma!(\sigma!(\mathbf{0})) \mid \sigma!(\text{for}(y \leftarrow \sigma)\mathbf{0})$$

binding u and v in P .

Fresh_2 is admissible: σ occurs in subject position and in the object of a lift, never under a written quote. After two administrative communications, $\{u, v\} = \{L(\sigma), R(\sigma)\}$, with the assignment chosen nondeterministically. That the assignment is nondeterministic is not a defect but the content of the design: only distinctness is required, never identity, which is exactly Lybech's criterion of parameter independence read as a design principle rather than as a proof obligation.

7.3 The clauses

The encoding maps a π term to an *address abstraction*, a function from a name to a rho process. We write $\llbracket P \rrbracket_{\sigma}$ for its value at σ .

Definition 7.4 (The translation).

$$\begin{aligned}
\llbracket \mathbf{0} \rrbracket_\sigma &= \mathbf{0} \\
\llbracket \bar{x}\langle z \rangle \rrbracket_\sigma &= x!(*z) \\
\llbracket x(y).P \rrbracket_\sigma &= \text{for}(y \leftarrow x) \llbracket P \rrbracket_\sigma \\
\llbracket P_1 \mid P_2 \rrbracket_\sigma &= \text{for}(m \leftarrow \sigma) \llbracket P_1 \rrbracket_m \mid \text{for}(m \leftarrow \sigma) \llbracket P_2 \rrbracket_m \mid \sigma!(\sigma!(\mathbf{0})) \mid \sigma!(\text{for}(y \leftarrow \sigma) \mathbf{0}) \\
\llbracket (\nu z)P \rrbracket_\sigma &= \text{Fresh}_2(\sigma; z, m). \llbracket P \rrbracket_m \\
\llbracket !x(y).P \rrbracket_\sigma &= \text{Fresh}_2(\sigma; a, b). \text{Fresh}_2(b; s, c). \left(D(a) \mid a!(B) \mid s!(*c) \right) \\
\text{where } B &= \text{for}(y \leftarrow x) \left(D(a) \mid \text{for}(c' \leftarrow s) \text{Fresh}_2(c'; d, e). (s!(*e) \mid \llbracket P \rrbracket_d) \right)
\end{aligned}$$

Top level: $\llbracket P \rrbracket \triangleq \llbracket P \rrbracket_{n_0}$ with $n_0 = @ \prod_{x \in \text{fn}(P)} x!(\mathbf{0})$.

Reading the replication clause. The node splits its own address twice, obtaining a box channel a , a local server port s , and a first seed c . The box B is posted on a and duplicated in the usual self-reproducing manner, so that $D(a) \mid a!(B) \rightarrow B \mid a!(B)$ and B then blocks on x : unfolding is input-guarded, and there is no divergence. On each unfolding, B reinstalls a duplicator, takes the current seed c' off the local port s , splits c' into a body address d and a successor seed e , posts e back on s , and runs the body at d .

The crucial point is what is *inside* the quote. When the box is posted, a, s, x have already been substituted, because they are bound by inputs whose continuations include the lift, and by Lemma 6.3 substitution reaches into a lift's object. Thereafter B is copied verbatim, and it needs no renaming, because it contains no addresses — only the port s on which it asks for one. This is Copy's first clause, $\bar{x} \perp (\bar{x}^l @ \bar{x}^r)$: the copies share the parent's port and the routing does the distinguishing.

Remark 7.5 (No global server). The port s is fixed and shared — among the copies of *one* replication node, and derived from *that node's* address. A term with k replication nodes has k mutually unaware local servers and no channel that all of them use. The scaffolding regress terminates at n_0 , which is computed by the translation from $\text{fn}(P)$ and consumed once. Compare Lybech's single $!N(x, z, v, s)$ on a fixed v .

Remark 7.6 (Whole-term compilation survives). Every process displayed above is written by the translation. What became dynamic is the *value* of an address, not the structure that computes it: the tree of Fresh_2 nodes is static and is isomorphic to the parse tree of the source. The encoding remains a whole-term compilation and remains non-homomorphic, which is the design (b) of §3.

7.4 Address allocation is a tree

Lemma 7.7 (Locality). *In $\llbracket P \rrbracket_{n_0}$, each address serves as the subject of the administrative traffic of exactly one clause occurrence, and the addresses allocated by distinct occurrences are distinct.*

Proof. Assign to each node of the source term the address at which its clause is invoked. The clauses for $\mathbf{0}$, output and input allocate nothing and pass their address to their continuation, so an address may be shared along a prefix chain but is consumed as a subject by at most the unique $|$, ν or $!$ node terminating that chain. Each of $|$, ν and Fresh_2 consumes its address σ as a subject and allocates exactly $L(\sigma)$ and $R(\sigma)$, which by Lemma 7.2 lie strictly below σ in

the address tree. Subtrees allocated at $L(\sigma)$ and $R(\sigma)$ are disjoint. For $!$, the successive seeds are $c_0 = c$ and $c_{k+1} = e_k$, where d_k, e_k are the two children of c_k ; hence the body of the k -th unfolding occupies the subtree below d_k , and these subtrees are pairwise disjoint and disjoint from every c_j . Note in particular that the successor seed is a *sibling* of the body address, not an extension of it; taking the successor seed to be $L(c_k)$ while the body ran at c_k would make the next seed collide with the body's own first child. $\square$

Corollary 7.8 (Distinctness). *All names generated by $\llbracket P \rrbracket_{n_0}$ are pairwise non- $\equiv_{\mathcal{N}}$, and none is $\equiv_{\mathcal{N}}$ to a name of $\text{fn}(P)$ or derivable from one.*

Proof. Lemmas 7.2 and 7.7; the last clause by the choice of n_0 , whose quote depth exceeds that of every $x \in \text{fn}(P)$. $\square$

Lemma 7.9 (Administrative reductions terminate). *Call a reduction administrative if its subject is an address. Every maximal administrative reduction sequence from $\llbracket P \rrbracket_{\sigma}$ is finite, and no administrative reduction is observable on $\varphi(\text{fn}(P))$.*

Proof. Each $|$ node contributes two, each ν node two, each $!$ node four plus, per unfolding, one on a and three on s and c_k . Unfoldings are guarded by an input on the source channel x , so no unfolding occurs without a corresponding source communication. Hence the number of administrative steps enabled at any point is bounded by a function of the number of source communications performed. Unobservability is Corollary 7.8. $\square$

Corollary 7.10 (Divergence reflection). $\llbracket P \rrbracket \rightarrow^{\omega}$ implies $P \rightarrow^{\omega}$.

8 The counterexamples, worked

We restate Examples 4.1 and 4.2 in the input-guarded fragment, guarding each replication with an input on a trigger channel w , and supplying triggers from the context.

Example 8.1 (Example 4.1, guarded).

$$P'_1 = !w(t).((\nu z)\bar{u}\langle z \rangle \mid Q) \quad P'_2 = (\nu z)!w(t).(\bar{u}\langle z \rangle \mid Q)$$

distinguished in π by $C = [\cdot] \mid \bar{w} \mid \bar{w} \mid u(n_1).u(n_2).(\bar{n}_1 \mid n_2.\bar{x})$.

In the image, the k -th unfolding of $\llbracket P'_1 \rrbracket$ runs its body at address d_k , and the body's parallel composition allocates $L(d_k), R(d_k)$ by communication on d_k , whose value is the k -th seed's child and therefore differs with k . The ν inside then allocates from an address that differs with k , so a different name is emitted on u at each unfolding. In $\llbracket P'_2 \rrbracket$ the ν is outside the box; its Fresh_2 fires once, and every unfolding emits the same z . $\llbracket C \rrbracket$ separates them. This is the failure of Example 4.1 removed: the offending static increment has become a communication, and the value it computes tracks the unfolding.

Example 8.2 (Example 4.2, guarded).

$$P_1 = !w(t).(\nu z)\bar{u}\langle z \rangle \quad P_2 = !w(t).(\nu q)(\nu z)\bar{u}\langle z \rangle, \quad P_1 \equiv P_2.$$

In $\llbracket P_1 \rrbracket$, unfolding k runs $\text{Fresh}_2(d_k; z, m).u!(*z)$, emitting one of $L(d_k), R(d_k)$. In $\llbracket P_2 \rrbracket$, unfolding k runs $\text{Fresh}_2(d_k; q, m).\text{Fresh}_2(m; z, m').u!(*z)$, emitting a child of m , itself a child of d_k . In both cases the emitted names are pairwise distinct across k , since the d_k are, and in neither case is any emitted name observable except as a π name. The two are therefore identified, as they must be. In the 2005 encoding they were separated because the extra ν performed a static increment that the replication context could not update; here it performs a communication that it can.

9 Full abstraction relative to encoded contexts

9.1 Context-labelled bisimilarity

Definition 9.1 (Context-labelled transitions). Let L be a calculus with reduction $\rightarrow$ and a class $\mathcal{C}$ of one-holed contexts. For $C \in \mathcal{C}$ write $P \xrightarrow{C} P'$ when $C[P] \rightarrow P'$, and $P \xRightarrow{C} P'$ for $C[P] \rightarrow^* P'$.

Definition 9.2 ($\mathcal{D}$ -relative bisimilarity). Let $\mathcal{D} \subseteq \mathcal{C}$. A symmetric relation $\mathcal{R}$ is a $\mathcal{D}$ -bisimulation if $P \mathcal{R} Q$ implies: for all $C \in \mathcal{D}$, if $P \xrightarrow{C} P'$ then $Q \xRightarrow{C} Q'$ for some Q' with $P' \mathcal{R} Q'$. Write $\approx_{\mathcal{D}}$ for the largest such relation.

On the source we take $\mathcal{D} = \mathcal{C}_{\pi}$, all π contexts, and write $\approx_{\pi}$. On the target we take $\mathcal{D} = \llbracket \mathcal{C}_{\pi} \rrbracket$, the *encodings of source contexts*, and write $\approx_{\llbracket \mathcal{C}_{\pi} \rrbracket}$.

Remark 9.3 (On minimality). The labels here are enabling contexts, not *minimal* enabling contexts. Minimality — relative pushouts in the sense of Leifer and Milner, and their refinement to a subcategory of admissible observers — is the right long-run discipline and is developed elsewhere; nothing in the theorem below needs it. Without minimality the label set is larger than necessary and $\approx_{\mathcal{D}}$ is correspondingly finer, which makes the completeness direction harder rather than easier. We prove it anyway and flag the refinement as future work.

Definition 9.4 (Encoding a context). $\llbracket - \rrbracket$ extends to contexts by $\llbracket [\cdot] \rrbracket_{\sigma} = [\cdot]_{\sigma}$, a hole expecting an address, and by the clauses of Definition 7.4 otherwise. Plugging is instantiation of the address abstraction: $\llbracket C[P] \rrbracket = \llbracket C \rrbracket [\llbracket P \rrbracket]$.

Lemma 9.5 (Compositionality). $\llbracket C[P] \rrbracket = \llbracket C \rrbracket [\llbracket P \rrbracket]$ for all π contexts C and processes P , on the nose.

Proof. By induction on C . Each clause of Definition 7.4 mentions its sub-translations only under an address abstraction, and the hole occurs in exactly one of them. $\square$

Note that this is Lybech's criterion 1 (compositionality) as a consequence, with the coordinating context being the **Fresh**₂ traffic and its free names contained in $\varphi(\text{fn}(P)) \cup \{\sigma\}$.

9.2 Address independence

Lemma 9.6 (Address independence). Let σ, τ be addresses satisfying Corollary 7.8 for P . Then there is a bijection β between the names generated by $\llbracket P \rrbracket_{\sigma}$ and those generated by $\llbracket P \rrbracket_{\tau}$, identity on $\varphi(\text{fn}(P))$, such that $\beta(\llbracket P \rrbracket_{\sigma}) = \llbracket P \rrbracket_{\tau}$; and consequently $\llbracket P \rrbracket_{\sigma} \approx_{\llbracket \mathcal{C}_{\pi} \rrbracket} \llbracket P \rrbracket_{\tau}$.

Proof. The first claim is the substitution $\theta(\sigma) \mapsto \theta(\tau)$, which is well defined and bijective by Lemma 7.2 and total on generated names by Lemma 7.7. For the second: β is not the identity, so the two terms are not equal; but an encoded context uses a name it has received only as the subject of an input or a lift, or as the object of one, and never applies $*$ to it or matches on its structure. Hence the only discriminations an encoded observer can make between two received names are equality tests by attempted synchronisation, which β preserves and reflects, being a bijection. $\square$

Lemma 9.6 is where the restriction to encoded observers first does work, and it should be read carefully: the statement is *false* for unrestricted rho observers, as §9.5 shows. This is the rho-calculus form of the π -calculus device of bisimulation up to a bijection on generated names.

9.3 Operational correspondence

Proposition 9.7 (Completeness). *If $P \rightarrow P'$ then $\llbracket P \rrbracket_\sigma \rightarrow^+ T$ with $T \approx_{\llbracket \mathcal{C}_\pi \rrbracket} \llbracket P' \rrbracket_{\sigma'}$ for some address σ' .*

Proof sketch. By induction on the derivation of $P \rightarrow P'$. For $[\pi\text{-COM}]$, $x(y).P_1 \mid \bar{x}\langle z \rangle \rightarrow P_1\{z/y\}$: the images are $\text{for}(y \leftarrow x)\llbracket P_1 \rrbracket_{\sigma_1}$ and $x!(*z)$ at sibling addresses, and one rho communication yields $\llbracket P_1 \rrbracket_{\sigma_1}\{z/y\}$, which equals $\llbracket P_1\{z/y\} \rrbracket_{\sigma_1}$ because y occurs in the image only in subject and object positions (this is the substitution proposition, and it is where admissibility is used a second time). Structural congruence: the interesting case is $P \mid \mathbf{0} \equiv P$, for which the images differ by two administrative steps and are related by Lemma 7.9 and Lemma 9.6. Replication: one unfolding of $!x(y).P$ is matched by the box's copy step, the seed exchange, and one **Fresh**₂, all administrative, followed by the communication on x . Restriction: **Fresh**₂, two administrative steps. The address bookkeeping is discharged by Lemma 9.6 throughout, which is what allows σ' to be an arbitrary address satisfying Corollary 7.8. $\square$

Proposition 9.8 (Soundness). *If $\llbracket P \rrbracket_\sigma \rightarrow^* T$ then $P \rightarrow^* P'$ for some P' with $T \approx_{\llbracket \mathcal{C}_\pi \rrbracket} \llbracket P' \rrbracket_{\sigma'}$.*

Proof sketch. By induction on the length of the reduction, classifying each step as administrative or as the image of a source communication (Lemma 7.9 makes the classification exhaustive, since every rho redex in the image has as subject either an address or a name of $\varphi(\text{fn}(P))$ or a ν -generated name, and the latter two are exactly the source channels). Administrative steps are absorbed into $\approx$: they are finite in number and produce no observable on $\varphi(\text{fn}(P))$. The only non-confluence among administrative steps is the **Fresh**₂ race, which permutes the two allocated addresses; the two outcomes are related by Lemma 9.6. $\square$

9.4 The theorem

Theorem 9.9 (Full abstraction). *For all π processes P, Q ,*

$$P \approx_\pi Q \quad \Longleftrightarrow \quad \llbracket P \rrbracket \approx_{\llbracket \mathcal{C}_\pi \rrbracket} \llbracket Q \rrbracket.$$

Proof sketch. ($\Rightarrow$) Let $\mathcal{R} = \{(\llbracket P \rrbracket, \llbracket Q \rrbracket) : P \approx_\pi Q\}$ closed under $\approx_{\llbracket \mathcal{C}_\pi \rrbracket}$ on both sides. Suppose $\llbracket P \rrbracket \xrightarrow{[C]} T$. If the step is administrative, T is related to $\llbracket P \rrbracket$ itself by Lemma 7.9, and $\llbracket Q \rrbracket$ matches by doing nothing. Otherwise, by Lemma 9.5, $\llbracket C \rrbracket[\llbracket P \rrbracket] = \llbracket C[P] \rrbracket$, and by Proposition 9.8 the step is the image of $C[P] \rightarrow P_1$; since $P \approx_\pi Q$ we have $C[Q] \rightarrow^* Q_1$ with $P_1 \approx_\pi Q_1$, and Proposition 9.7 supplies the matching target moves.

($\Leftarrow$) Contrapositively, let C separate P from Q in π . Then $\llbracket C \rrbracket$ is a label in $\llbracket \mathcal{C}_\pi \rrbracket$, and by Lemma 9.5 and Proposition 9.7 the separating source computation is reproduced in the image; the observable distinguishing $C[P]$ from $C[Q]$ is a barb on a name of $\varphi(\text{fn}(P) \cup \text{fn}(Q))$, and barbs on such names are preserved and reflected by Lemma 7.9 and Corollary 7.8. Hence $\llbracket P \rrbracket \not\approx_{\llbracket \mathcal{C}_\pi \rrbracket} \llbracket Q \rrbracket$. $\square$

9.5 The restriction is forced

Proposition 9.10 (Unrestricted observers are too strong). *There are π processes P, Q with $P \equiv Q$ — hence $P \approx_\pi Q$ — and a rho context C , not in $\llbracket \mathcal{C}_\pi \rrbracket$, with $C[\llbracket P \rrbracket] \not\approx C[\llbracket Q \rrbracket]$.*

Proof. Take $P = \bar{u}\langle v \rangle$ and $Q = \bar{u}\langle v \rangle \mid \mathbf{0}$. Then $\llbracket P \rrbracket_{n_0} = u!(*v)$, which has no barb on n_0 , whereas

$$\llbracket Q \rrbracket_{n_0} = \text{for}(m \leftarrow n_0) u!(*v) \mid \text{for}(m \leftarrow n_0) \mathbf{0} \mid n_0!(n_0!(\mathbf{0})) \mid n_0!(\text{for}(y \leftarrow n_0) \mathbf{0})$$

has a barb on n_0 . Since n_0 is computed from $\text{fn}(P)$ by the translation, an arbitrary rho context can name it; take $C = [\cdot] \mid \text{for}(k \leftarrow n_0) \text{succ}!(\mathbf{0})$. No encoded context can, because $n_0 \notin \varphi(\text{fn}(P))$ by construction — which is precisely the content of the restriction. $\square$

Remark 9.11 (A sharper form of the same phenomenon). The example above exploits the encoding’s scaffolding. A second, deeper form exploits reflection itself: an arbitrary rho context may apply $*$ to a name received on u , running the process the name quotes. Since a ν -generated name in our encoding is $L(\sigma)$ or $R(\sigma)$ for an internal address σ , dropping it executes $\sigma!(\mathbf{0})$ or $\text{for}(y \leftarrow \sigma) \mathbf{0}$ and exposes a barb on an internal address. No encoded context does this: an encoded context uses a received name only as a π name, i.e. in subject or object position. This is not a weakness of the present encoding. By Lybech’s separation theorem, the rho calculus manufactures new *free* — hence observable and substitutable — names at runtime, and no encoding of π into rho, his or ours, can be fully abstract against unrestricted rho observers. The restriction to $\llbracket \mathcal{C}_\pi \rrbracket$ is the shape of the theorem, not a weakening of it.

9.6 Comparison with the $\mathcal{N}$ -restricted criteria

Proposition 9.12. *Theorem 9.9 implies Lybech’s criteria 1–6 for $\llbracket - \rrbracket$, with $\simeq$ taken to be $\approx_{\llbracket \mathcal{C}_\pi \rrbracket}$ and the name derivation function δ taken to be $\sigma \mapsto \{L(\sigma), R(\sigma)\}$.*

Proof sketch. Compositionality is Lemma 9.5; substitution invariance is the substitution proposition used in Proposition 9.7; operational correspondence is Propositions 9.7 and 9.8; observational correspondence follows from Corollary 7.8 and Lemma 7.9, since the observable names are exactly $\varphi(\text{fn}(P))$ and administrative traffic is confined to addresses; divergence reflection is Corollary 7.8’s companion; and parameter independence is Lemma 9.6. $\square$

Remark 9.13. Lybech’s $\approx^\mathcal{N}$ restricts observation by *stipulating* a set of names. Ours restricts it by *deriving* the observers from the source: $\mathcal{D} = \llbracket \mathcal{C}_\pi \rrbracket$. The two agree here, because π and rho are both name-passing calculi and barbs are an adequate notion of observation for both. They do not agree in general, and a presentation in terms of a name set assumes that what one can observe of a term is the set of names it can communicate on — true for these two calculi and false for, e.g., the ambient calculus.

10 Comparison and discussion

	MR 2005	Lybech 2022	this note
runtime name source	none	one, global	none
topology of name traffic	—	star	tree, $\cong$ parse tree
whole-term compilation	yes	no	yes
replication	eager, diverges	input-guarded	input-guarded
stated invariant	none	most-recent params	prefix-incompatibility
correctness criterion	full abstraction (false)	six criteria	full abstraction rel. $\llbracket \mathcal{C}_\pi \rrbracket$
status	refuted	proved	this paper

Three remarks on what has and has not been achieved.

Freshness is confined, not eliminated. Abramsky’s construction is not free of a freshness assumption either: his name convention stipulates that a different root is introduced at each instance of the Axiom, With and Of Course rules. What the construction buys — and what this note inherits — is that freshness is required only *statically*, finitely often, at translation time. At runtime there is no allocation: only address extension, performed locally, by communication. That is the defensible form of the claim that the rho calculus needs no theory of names from outside, and it is strictly what the name server costs.

Demand-driven copying is a dividend, not a patch. Abramsky’s Copy fires only when a contraction asks for a second copy. Restricting to input-guarded replication — which both Lybech and this note do, to avoid divergence — is the process-calculus shadow of that discipline. It is worth recording that the fix for divergence and the fix for freshness come from the same place.

Linearity was doing work that we must now do by hand. In PE_2 the address invariant is protected by linearity (each name occurs exactly twice) and acyclicity (which Abramsky uses to guarantee deadlock freedom). The rho calculus has neither: names are used by arbitrarily many senders and receivers. What was structural there is a theorem here, and its proof is Lemmas 7.2 and 7.7, which decompose along exactly Abramsky’s two axes — depth and siblings.

11 Related work

Abramsky’s programme was carried forward by Bellin and Scott [2], who translate classical linear logic proof structures into a synchronous π -calculus, and tightened into a session-typed discipline by Caires and Pfenning [3] and Wadler [14]. Kokke, Montesi and Peressotti [8] observe that the separation of environments internalised by $\otimes$ corresponds to parallel process behaviour, and extend linear logic to hyperenvironments accordingly; that is the same move as the present one, made from the side of the logic rather than the side of the encoding.

On encodability, Gorla [6] supplies the criteria Lybech adapts, and Gorla and Nestmann [7] argue against full abstraction as a criterion; van Glabbeek [5] derives validity from a semantic equivalence rather than a list, which is closer in spirit to what we do here, though he does not treat parametrised translations. Sangiorgi’s encoding of $\text{HO}\pi$ into π [13] is the result Lybech’s separation theorem shows does not extend to reflection. Bendixen, Bojesen, Hüttel and Lybech [12] instantiate the rho calculus as a higher-order Ψ -calculus. Leifer and Milner [9] supply the derived-transition machinery that a minimal-label refinement of §9 would use.

12 Open problems

1. **Minimal labels.** Replace enabling contexts by minimal ones (relative pushouts), taken relative to the observer class $\llbracket \mathcal{C}_\pi \rrbracket$ rather than to the whole of rho, and show that Theorem 9.9 survives with a coarser relation.

2. **Full proofs.** Propositions 9.7, 9.8, 9.12 and Theorem 9.9 are given as sketches. The bookkeeping is routine but long, and the case that deserves scrutiny is the interaction of the **Fresh₂** race with replication.
3. **Polyadic π and joins.** The encoding is monadic. Polyadic synchronisation should be encoded using the join form $\text{for}(y_1 \leftarrow x_1 \ \& \ y_2 \leftarrow x_2)P$ directly, which would put §2.1(2) to work and connect with Carbone and Maffei [4].
4. **Choice.** The source fragment is choice-free.
5. **Cost.** The **Fresh₂** tree is a metering surface: each source ν costs two communications, each unfolding four. What the encoding charges is a natural object, and it is not treated here.
6. **Mechanisation.** Definition 6.6 is a syntactic side condition on translation clauses and should be machine checked; a translation that satisfies it cannot have a bug of the kind found in 2022.

Acknowledgements

The author thanks Stian Lybech, whose counterexamples are correct and whose diagnosis of their common cause is the reason the present repair could be stated so briefly. Claude (Anthropic) assisted with source review, drafting, and proof-checking.

References

- [1] S. Abramsky. *Computational interpretations of linear logic*. Theoretical Computer Science 111(1–2):3–57, 1993.
- [2] G. Bellin and P. J. Scott. *On the π -calculus and linear logic*. Theoretical Computer Science 135(1):11–65, 1994.
- [3] L. Caires and F. Pfenning. *Session types as intuitionistic linear propositions*. CONCUR 2010, LNCS 6269, 222–236.
- [4] M. Carbone and S. Maffei. *On the expressive power of polyadic synchronisation in π -calculus*. Nordic Journal of Computing 10(2):70–98, 2003.
- [5] R. van Glabbeek. *A theory of encodings and expressiveness*. FoSSaCS 2018, 183–202.
- [6] D. Gorla. *Towards a unified approach to encodability and separation results for process calculi*. Information and Computation 208(9):1031–1053, 2010.
- [7] D. Gorla and U. Nestmann. *Full abstraction for expressiveness: history, myths and facts*. Mathematical Structures in Computer Science 26:639–654, 2014.
- [8] W. Kokke, F. Montesi and M. Peressotti. *Better late than never: a fully abstract semantics for classical processes*. Proc. ACM Program. Lang. 3(POPL), 2019.
- [9] J. J. Leifer and R. Milner. *Deriving bisimulation congruences for reactive systems*. CONCUR 2000, LNCS 1877, 243–258.

- [10] S. Lybech. *Encodability and separation for a reflective higher-order calculus*. EXPRESS/SOS 2022, EPTCS 368, 95–112. Technical report: Reykjavík University, 2022.
- [11] L. G. Meredith and M. Radestock. *A reflective higher-order calculus*. Electronic Notes in Theoretical Computer Science 141(5):49–67, 2005.
- [12] A. R. Bendixen, B. B. Bojesen, H. Hüttel and S. Lybech. *A generic type system for higher-order Ψ -calculus*. EXPRESS/SOS 2022, EPTCS 368, 43–59.
- [13] D. Sangiorgi. *From π -calculus to higher-order π -calculus — and back*. TAPSOFT 1993, LNCS 668, 151–166.
- [14] P. Wadler. *Propositions as sessions*. Journal of Functional Programming 24(2–3):384–418, 2014.

A Status of results

Result	Status	Note
Prop. 6.1, 6.2	proved	elementary
Lemma 6.3, Cor. 6.4	proved	
Prop. 6.5	proved	
Lemma 7.2	proved	uses Lybech’s stratification lemma
Lemma 7.7, Cor. 7.8	proved	
Lemma 7.9	proved	
Lemma 9.5	proved	
Lemma 9.6	proved	the $\mathcal{D}$ -relativity is essential
Prop. 9.7, 9.8	sketch	bookkeeping routine; see open problem 2
Thm. 9.9	sketch	modulo the above
Prop. 9.10	proved	
Prop. 9.12	sketch	modulo operational correspondence

B The encoding in rholang

```
// Fresh2(sigma; u, v).P --- two addresses from one, by communication.
// L(sigma) = @{ sigma!(Nil) }   R(sigma) = @{ for (y <- sigma) { Nil } }

// Fresh2(sigma; u, v).P expands inline to:
//
//   for (u <- sigma) { for (v <- sigma) { P } }
//   | sigma!( sigma!(Nil) )
//   | sigma!( for (y <- sigma) { Nil } )
//
// After two communications, {u,v} = { L(sigma), R(sigma) }.

// [[ P1 | P2 ]]_sigma
//   for (m <- sigma) { [[P1]]_m } | for (m <- sigma) { [[P2]]_m }
//   | sigma!( sigma!(Nil) ) | sigma!( for (y <- sigma) { Nil } )
```

```

// [[ (nu z) P ]]_sigma
//   for (z <- sigma) { for (m <- sigma) { [[P]]_m } }
//   | sigma!( sigma!(Nil) ) | sigma!( for (y <- sigma) { Nil } )

// [[ ! x(y).P ]]_sigma --- local address server on s, box on a
//   Fresh2(sigma; a, b). Fresh2(b; s, c).
//   ( D(a) | a!( B ) | s!( *c ) )
//   B = for (y <- x) {
//     D(a) |
//     for (cp <- s) {
//       Fresh2(cp; d, e).( s!(*e) | [[P]]_d )
//     }
//   }
//   D(a) = for (q <- a) { *q | a!(*q) }

```

The listing is schematic in one respect: rholang's receipts are linear-or-persistent as a whole, and the two-message handshake of **Fresh**₂ is written above with two separate linear receives on the same channel, which is what the calculus prescribes. A production implementation would use the join form, `for (u <- s & v <- s) { ... }`, which consumes both in one rendezvous and removes the race of §7 altogether — at the cost of assuming joins, which the pure calculus does not have. This is the concrete payoff of §2.1(2) and is recorded here rather than assumed in the theory.